What is the helpful need in contractList for the contructor and getters

Analyzed ContractList structure and identified missing public getter methods >

Looking at `ContractList.java` as it stands:

Constructor:

```java
public ContractList() {
    contractCollection = new ArrayList<>();
}
```

Its only job is to initialize the empty `ArrayList<Contract>`. That's it — no parameters, no special setup logic.

Getters:
Here's the gap — `ContractList` **currently has no public getters at all.** The only lookup method is:

```java
private Contract findContract(String contractId)
```

and it's `private`, so nothing outside `ContractList` can use it.

That matters because right now no other class can query contract data. For example:

- `SalaryList` checks `playerList.findPlayerById(...)` before paying a player, but it has **no way to check if that player has an active contract** — because `ContractList` doesn't expose anything.
- If you ever want a rule like "can't add a sal... record unless the player has an active contract," `SalaryList` would need something like:

- `SalaryList` checks `playerList.findPlayerById(...)` before paying a player, but it has **no way to check if that player has an active contract** — because `ContractList` doesn't expose anything.
- If you ever want a rule like "can't add a salary record unless the player has an active contract," `SalaryList` would need something like:

```java
public Contract findContractByPlayerId(String playerId) {
    for (Contract contract : contractCollection) {
        if (contract.getPlayerId().equals(playerId)) {
            return contract;
        }
    }
    return null;
}
```

or even more directly:

```java
public boolean hasActiveContract(String playerId) {
    Contract c = findContractByPlayerId(playerId);
    return c != null && c.getContractStatus().equals("active");
}
```